



TECHNICAL GUIDEBOOK · INTERNAL & NDA PARTNERS

Confidential · 2026 Edition



NINE AGENTS · ONE PIPELINE · ZERO BUSYWORK

CONFIDENTIAL — DISTRIBUTION RESTRICTED.

This document contains the proprietary architecture, scoring models, and agent designs of the L3xy platform. Access is limited to L3xy employees, contractors, and partners who have executed a Mutual Non-Disclosure Agreement. Do not forward, copy, or share without written approval from L3xy Inc.

1. Executive summary

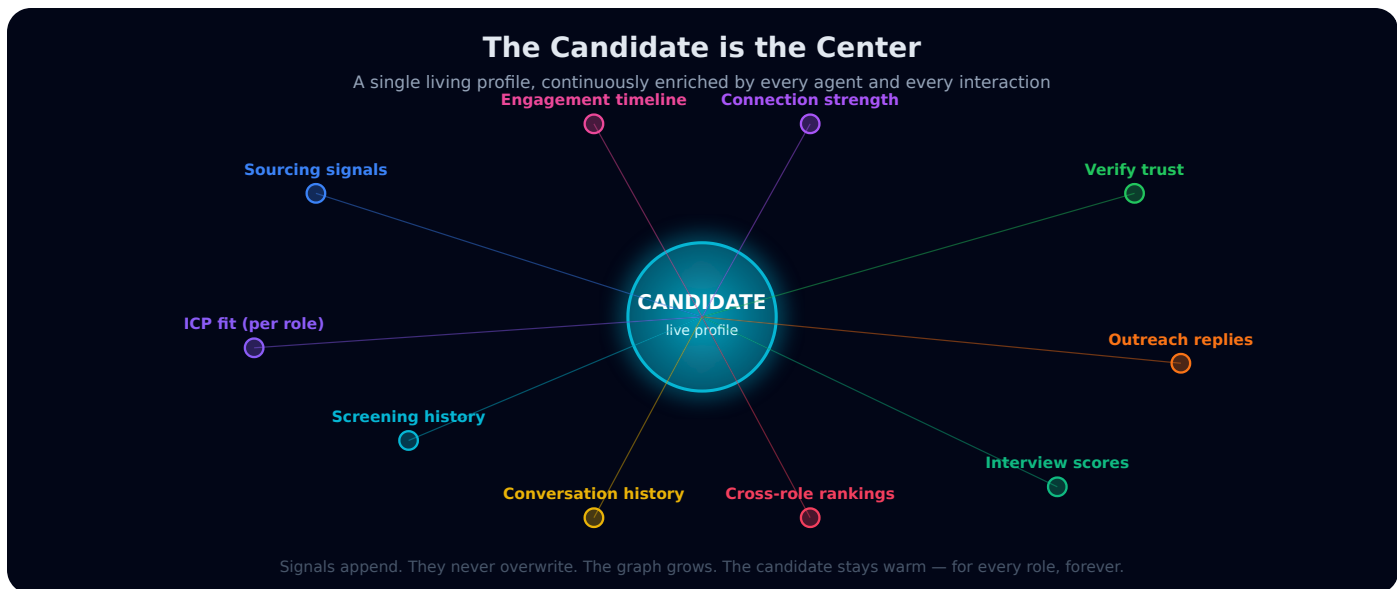
L3xy is a multi-tenant, autonomous AI hiring platform built around a **living candidate graph**. It coordinates a team of nine specialized AI agents through a Bayesian intelligence engine that scores, ranks, and recommends actions for every candidate in real time — and re-scores every candidate against every open role continuously, not just at sourcing time.

The product replaces the manual recruiter workflow (sourcing → screening → outreach → scheduling → interviewing) with a single self-running pipeline that the recruiter steers rather than executes. Every signal the agents produce flows into a composite scoring model that outputs a hire probability, a “next best action,” and a full decision audit trail.

The architectural distinction that matters most: **L3xy is not an ATS-on-AI**. ATS systems are write-once databases of historical candidate state. L3xy is a real-time graph where candidate signals decay, refresh, cross-pollinate across roles, and continuously re-rank without recruiter intervention. The talent pool is a *compounding asset* — every interaction makes the next decision sharper.

This guidebook documents the architecture, the math, the data sources, the agent designs, and the closed-loop learning system that the public overview deck only references.

1.1 The Living Talent Graph (architectural contract)



The Living Talent Graph — every candidate is a node that grows richer over time

Every candidate row in `candidates` is the root node of a graph that includes:

- **Signal vector** (`signalsJson`): append-only log of every agent output, each tagged with `agentId` , `producedAt` , `confidence` , and decay class (§4.2).
- **Engagement timeline** (`connection_events`): every reply, no-show, schedule change, voluntary action — feeds the Connection Strength engine (§5).
- **Role-fit cache**: per-job ICP match scores, recomputed on (a) new agent signal arrival, (b) job ICP change, © policy update, or (d) decay-threshold crossing.
- **Conversation history** across all channels, indexed by candidate, not by job.

Three architectural rules enforce the “living” property:

1. **No destructive writes.** Agents append; they never overwrite. A re-screen six months later doesn’t erase the original screening — it adds a new signal with its own timestamp, and the engine decides which dominates based on freshness × confidence.
2. **Decay is read-time, not write-time.** Signals are stored at full strength; their effective weight is computed when the score is read. This means a tenant that adjusts decay constants doesn’t have to backfill — the next read is correct.
3. **Cross-role re-evaluation is automatic.** When a new job’s ICP is published, the role-fit cache invalidates and the next pipeline launch re-ranks every candidate in the tenant’s pool against the new ICP — without re-sourcing them externally. This is the operational definition of “warm pool.”

2. System architecture

2.1 High-level topology

L3xy is a pnpm monorepo with the following deployable artifacts:

Artifact	Purpose
<code>artifacts/lexy</code>	Recruiter web app (React + Vite, cyan/dark <code>#06b6d4</code> / <code>#050a0f</code> theme)
<code>artifacts/lexy-site</code>	Public marketing site
<code>artifacts/lexy-demo</code>	Sandboxed demo tenant for prospects
<code>artifacts/api-server</code>	Express API server, agent orchestrator, intelligence engine
<code>lib/db</code>	Drizzle ORM schema + Postgres migrations (shared)

2.2 Multi-tenant model

- Every row in every business table carries `tenant_id`.
- `resolveUser` middleware enforces tenant isolation on every request.
- Roles: `platform_admin`, `tenant_admin`, `recruiter`, `hiring_manager`.
- Tenant-level policy objects govern automation thresholds (see §6).

2.3 Data layer

Postgres with Drizzle ORM. Notable schema decisions:

- `candidates.email` is `NOT NULL` with a partial unique index on `(tenant_id, lower(email))` `WHERE email IS NOT NULL` to allow placeholder emails for resume-only candidates while still enforcing dedup on real ones.
 - `pipeline_runs` is the source of truth for agent execution; every stage transition is appended as a row in `pipeline_run_stages` with `status`, `startedAt`, `completedAt`, and `outputJson`.
 - The intelligence engine reads agent outputs through `signalsJson` columns rather than re-running expensive AI calls.
-

3. Agent inventory and execution order

The orchestrator (`artifacts/api-server/src/lib/agents/orchestrator.ts`) coordinates nine production agents in canonical execution order.

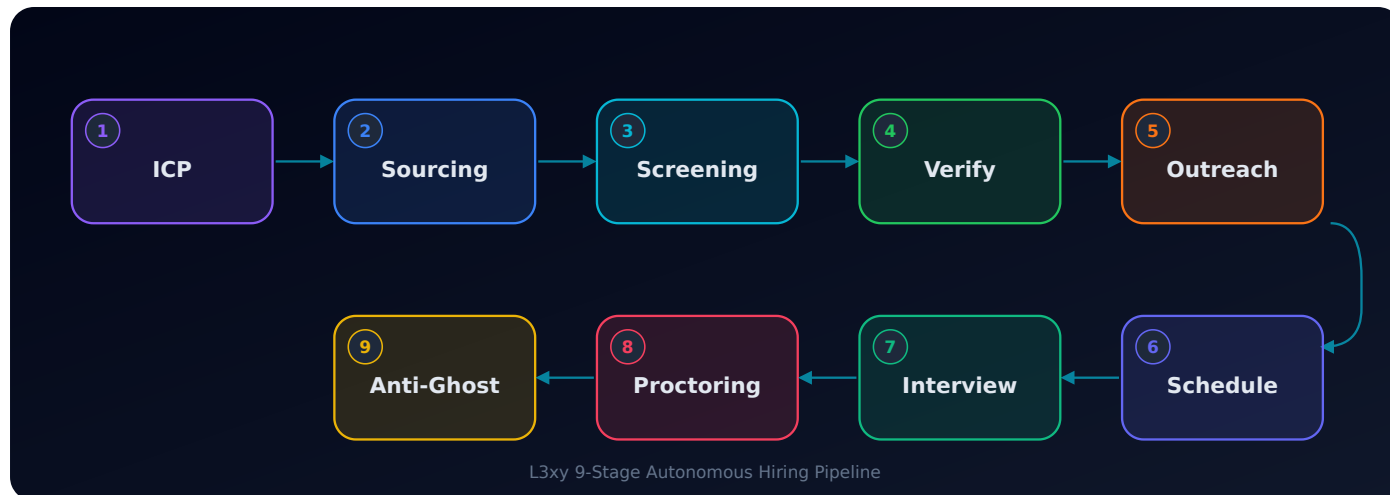


Figure 1 — The L3xy 9-stage autonomous hiring pipeline

3.1 Canonical AGENT_ORDER

```
icp(1) → sourcing(2) → screening(3) → verification(4) → outreach(5)  
→ scheduling(6) → interview(7) → proctoring(8) → anti-ghosting(9)
```

`analytics` exists as a registered agent (order 99) but is intentionally not part of the runnable pipeline — it is a results dashboard surfaced under the Intelligence tab. Legacy configs that include it are stripped on load.

3.2 Dependency graph (UI enforcement)

The frontend `WorkflowCanvas` enforces this DAG when the recruiter toggles agents on/off; selecting any downstream agent automatically pulls in its prerequisites with a toast notification, and disabling an upstream agent cascades to its dependents.

Agent	Requires
sourcing	icp
screening	sourcing
verification	screening
outreach	screening
scheduling	outreach
interview	scheduling
proctoring	interview
anti-ghosting	outreach

3.3 Per-agent design

ICP AGENT

Parses the job description into a structured `IdealCandidateProfile` : required skills, preferred skills, seniority band, industry domain, and weighted attributes. Output is canonical input for sourcing and screening. Uses an LLM with a strict JSON schema and a deterministic temperature.

SOURCING AGENT

The Sourcing agent is **pool-first, external-last**. This is the operational expression of the Living Talent Graph: every external API call is the most expensive thing the platform can do, and most of the time it’s unnecessary because the right candidate is already inside L3xy.

Sourcing executes a strict three-tier waterfall. Each tier must complete and fail to satisfy the target candidate count before the next tier is invoked.

Tier 1 — Tenant Talent Pool (zero cost, zero latency)

The first query goes to the tenant’s own `candidates` table, ranked against the new job’s ICP via the role-fit cache (§1.1). The Talent Match service (`talent-match.ts`) runs the new ICP vector against every candidate in the tenant — including those previously evaluated for other roles, those who responded “not now,” and those at any stage of any prior pipeline.

For each tenant-pool match, the engine:

- Recomputes role-fit using the *current* ICP (not the cached score from the original role)

- Applies decay weighting to all stored signals (§4.2) — fresh signals dominate
- Surfaces the candidate's prior conversation history, prior outreach state, and DNC flags
- Carries forward verification status and trust signals (no need to re-verify)

If Tier 1 produces enough qualified candidates above the ICP threshold to satisfy the work order's target count, **the pipeline stops sourcing here**. No external API is called. The recruiter sees a fully populated pipeline of warm, pre-known humans within seconds.

Tier 2 — Platform Talent Pool (zero external cost, opt-in)

If Tier 1 is short, the agent queries the cross-tenant **Platform Pool** — the federated index of candidates across all tenants who have explicitly opted into platform-wide visibility (or whose tenant has enabled the cross-org talent network). Governance rules:

- A candidate's identity, contact data, and conversation history are **never** exposed cross-tenant. Tier 2 surfaces a *fit profile* — skills, experience, anonymized signal summary — and produces an introduction request that the candidate must approve before identity is unblinded to the requesting tenant.
- Tenant policy (`platform_pool_share` , `platform_pool_consume`) controls participation in each direction independently.
- Candidates who opt out at the candidate level are excluded from Tier 2 results regardless of tenant settings.
- Every Tier 2 surface is logged with the consent record that authorized it (auditable).

The platform pool gets larger and more useful with every tenant on the network — the classic two-sided talent flywheel. The first tenant to integrate gets the smallest pool. The hundredth tenant gets a pool that no external provider can match.

Tier 3 — External Discovery (paid, last resort)

Only if Tiers 1 and 2 *combined* fall short of the target does the agent reach for paid external sources. Tier 3 itself is a two-phase architecture:

- **Phase 3a (parallel discovery):** GitHub API, People Data Labs (PDL), and SerpAPI are queried concurrently. PDL queries use Elasticsearch DSL; SerpAPI queries use boolean LinkedIn search strings; GitHub queries hit `/search/users` filtered by language, followers, repo count.
- **Phase 3b (enrichment):** EnrichLayer ingests Phase-3a LinkedIn URLs and resolves them into deep profile records (titles, companies, skills). The enrichment cap is applied on the **output** side, not the input side, to avoid wasting budget on profiles that fail enrichment.

PDL profiles are flattened (`skills.name` → `skills` field) before scoring. `enrichOneProfile` logs every failure with the upstream URL for debuggability.

Why this matters architecturally

Every Tier 3 candidate that passes screening is *immediately* added to the tenant's pool (Tier 1) and — if tenant policy allows — to the platform pool (Tier 2). The platform's marginal sourcing cost approaches zero over time. By design, a tenant that has been on L3xy for 12 months should be servicing 60-80% of new role demand from Tiers 1 and 2 alone. The external API budget is reserved for genuinely novel skills and emerging markets.

This is the inversion of every other “AI sourcing” tool: they spend more on external APIs every quarter as their tenant grows. L3xy spends *less*.

SCREENING AGENT

Resume + profile + ICP → match percentage, gap analysis, structured recruiter summary. Two-pass LLM: first pass extracts structured experience, second pass scores against the ICP using a deterministic rubric. Outputs feed both the candidate Intelligence Card and the bulk pipeline view.

VERIFY AGENT

Cross-checks identity using:

- LinkedIn URL match (canonical handle vs. resume handle)
- Resume-to-profile consistency check (employer overlap, date overlap)
- Disposable email and burner phone detection (regex + provider lookup)
- Optional ID-document upload review (manual escalation path)

The orchestrator gate refuses to run Verify if Screening hasn't produced eligible candidates (“No candidates in screening stage ready for verification”).

OUTREACH AGENT

Multi-step sequence engine. For each candidate:

- Generates personalized first-touch copy using ICP fit reasons
- Selects optimal send time per recipient timezone
- Maintains conversation state across replies
- Hands off to Anti-Ghost when silence thresholds are crossed

SCHEDULE AGENT

Calendar invite generation, reminder scheduling, timezone normalization. Issues unique interview-room URLs per session. Integrates with the Anti-Ghost agent for reminder cadence.

INTERVIEW AGENT

Conducts AI-led video interviews. Four sub-types share a base agent with track-specific question banks:

- **Behavioral** — STAR-method evaluation
- **Cultural** — tenant-supplied cultural document grounds the questions
- **Technical** — adaptive difficulty (easy / medium / hard / mixed)
- **Programming** — language-aware coding rounds

Eligibility gate (Gate 2 inside `_runInterview`) rejects manual candidates whose `verification_status` is not `"verified"` — enforced because manual uploads can include unvetted profiles.

PROCTORING AGENT

Real-time vision pipeline during the interview session:

- Multi-face detection
- Gaze-direction estimation (off-screen %)
- Tab/visibility-change instrumentation
- Background audio anomaly flags

Outputs an integrity score that feeds the Trust composite.

ANTI-GHOST AGENT

Implemented in `anti-ghost-engine.ts` and driven by `anti-ghost-scheduler.ts` . Runs **four parallel detectors every 30 minutes** plus a **nurture cycle every 6 hours**:

Detectors (every 30 min):

1. `detectInterviewNoShows` — interviews past their scheduled time with no completion signal
2. `detectOutreachDropouts` — candidates who replied earlier in a sequence then went silent
3. `detectStalePipeline` — applications stuck in early stages for 21+ days
4. `detectOfferLimbo` — offers pending for 7+ days without acceptance

Tiered re-engagement (within active pipelines):

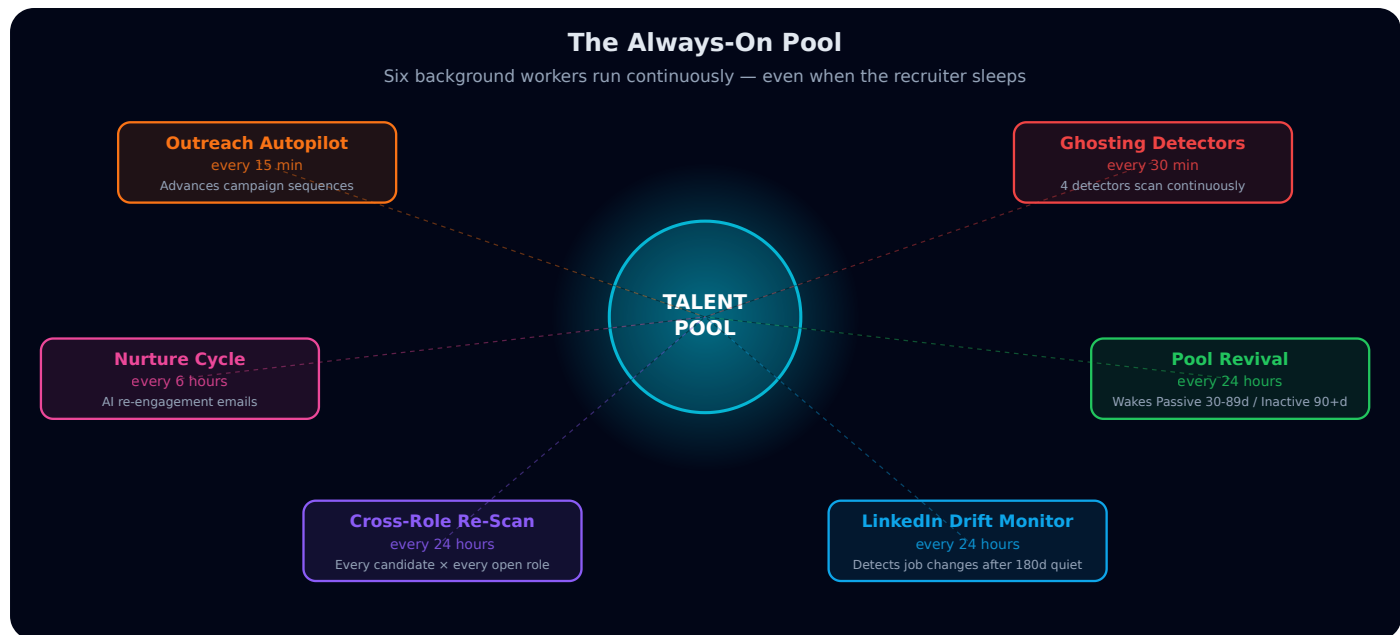
1. Auto-friendly nudge (24h silence)
2. Value-add follow-up (72h)
3. Recruiter escalation with one-click handoff (>5 days)

Nurture cycle (every 6h) — `processNurtureCycle()` moves flagged candidates into the `nurture_pool` table and generates AI re-engagement emails via `generateReEngagementEmail()` with configurable tone (warm / professional / brief). The nurture pool is the conveyor belt that returns dormant candidates to active engagement without losing them.

Decay-aware throughout so the agent doesn't pester candidates who responded recently on another channel.

3.4 The Living Pool — always-on background operations

The agents above run *on demand* when a recruiter launches a pipeline. The Living Pool runs *continuously* in the background, regardless of whether anyone is using the product. This is what makes the talent graph *alive* rather than archival.



Six always-on background workers keep the pool alive 24/7

Six scheduled workers run permanently inside the api-server (`src/index.ts`):

Scheduler	File	Cadence	Purpose
Outreach Scheduler	<code>outreach-scheduler.ts</code>	every 15 min	Drives multi-step campaign sequences; sends the next nudge when reply windows lapse
Anti-Ghost Detectors	<code>anti-ghost-scheduler.ts</code>	every 30 min	Runs the four detectors; flags new ghosting risks for the nurture cycle
Anti-Ghost Nurture Cycle	<code>anti-ghost-scheduler.ts</code>	every 6 h	Processes the nurture pool; generates and sends AI re-engagement emails
Candidate Re-engagement	<code>candidate-reengagement-scheduler.ts</code>	every 24 h	Scans the entire tenant pool; categorizes candidates by inactivity (Passive 30-89d / Inactive 90+d) and sends “are you still looking?” mailers
Platform Recommendation Scan	<code>platform-recommendation-scheduler.ts</code>	every 24 h	Re-evaluates every platform-pool candidate against every open work order across all tenants (cross-role re-ranking); strong matches (≥ 75) trigger a candidate-facing notification
LinkedIn Drift Monitor	<code>linkedin-profile-monitor.ts</code>	every 24 h	For any candidate whose Lexy profile is 180+ days old, fetches live LinkedIn data via Enrich Layer, runs <code>detectJobChange()</code> against <code>currentTitle</code> / <code>currentCompany</code> , and sends either a job-change congratulations email or a 6-month wellness check-in fallback

RE-ENGAGEMENT CATEGORIZATION

The 24-hour `candidate-reengagement-scheduler` reads from `candidates` and bins each one by last activity timestamp:

Category	Last activity	Treatment
Active	<30 days	No action — they’re already in a live conversation or pipeline
Passive	30-89 days	<code>buildPassiveEmail()</code> — soft, value-add re-engagement
Inactive	90+ days	<code>buildInactiveEmail()</code> — explicit “are you still open to opportunities?” check-in
DNC	any	Excluded — Do Not Contact flag is honored cross-role and forever

Each send writes a new event to `connection_events`, which feeds back into the Connection Strength engine (§5) — so even a “no thanks” reply becomes a fresh signal that updates the candidate’s engagement profile.

CROSS-ROLE RE-RANKING (THE AUTO-PROMOTION LOOP)

`platform-recommendation-engine.ts → runPlatformRecommendationScan()` runs every 24 hours and:

1. Loads every open work order across the platform pool tenants where `platformRecommendationsEnabled` is true.
2. For each candidate in the pool, runs an LLM-driven fit evaluation against every open job (GPT-4o rubric).
3. Materializes high-confidence matches (above the per-tenant `recommendation_threshold`) into the recruiter’s Recommended Actions queue.
4. **Reverse-pitches the candidate.** When the score crosses ≥ 75 , `sendMatchNotificationEmail()` automatically emails the candidate: *“A company has shown interest in your profile — our AI matching engine identified you as a strong fit for a [Job Title] role.”* Cooldowns and DNC are honored.
5. Logs the match decision with the policy snapshot and the ICP version that produced it (auditable).

This is the operational mechanism behind the Living Talent Graph promise: a candidate who said “not now” in March doesn’t disappear — within 24 hours of a fitting role being opened, the recruiter sees them as a Recommended Action *and* the candidate gets a heads-up email asking if they want the intro. **The platform sells opportunities to the candidate, not just candidates to the recruiter.**

LINKEDIN DRIFT DETECTION (THE 6-MONTH WELLNESS LOOP)

`linkedin-profile-monitor.ts → runLinkedInProfileMonitor()` is the answer to “how do you know if a candidate changed jobs and never told you?” It runs every 24 hours and operates as follows:

1. Selects platform-pool candidates whose Lexy profile is older than `STALE_THRESHOLD_DAYS = 180`.
2. Honors a 90-day per-candidate cooldown (read from `communicationEventsTable`) to prevent over-contact.
3. For each eligible candidate, calls `fetchLinkedInViaEnrichLayer()` to retrieve their current LinkedIn profile (if a `linkedinUrl` is on file).
4. `detectJobChange()` compares the live `experience[0]` title + company against `candidates.currentTitle` / `currentCompany`.
5. **Branch:**

- If a job change is detected → `buildJobChangeEmail()` sends a personalized “congrats on the new role at [Company] — want to update your Lexy profile?” mailer.
 - If no change (or Enrich Layer data isn’t available) → `buildCheckInEmail()` sends the calendar-based wellness check: *“Has something exciting happened lately? New job, promotion, relocation, certification?”*
6. All sends are written to `communicationEventsTable` with the appropriate `re_engagement` / `follow_up` event type, which both updates the engagement score and resets the cooldown.

The result: a candidate who got promoted six months ago and forgot about Lexy gets a friendly congratulations email, an effortless one-click profile refresh path, and the recruiter gets a re-scored, up-to-date profile without lifting a finger.

WHY THIS IS THE *REAL* MOAT (OPERATIONALIZED)

A static ATS can claim “we keep your candidates.” A pipeline tool can claim “we send sequences.” Neither runs the loop that closes ghosting risk → nurture revival → cross-role re-evaluation → automatic re-surface, all on independent cadences, all without a human in the loop.

L3xy’s pool stays warm because the system never sleeps — even when the recruiter does.

4. The Intelligence Engine

Lives in `artifacts/api-server/src/lib/intelligence.ts`. This is the core of the platform.

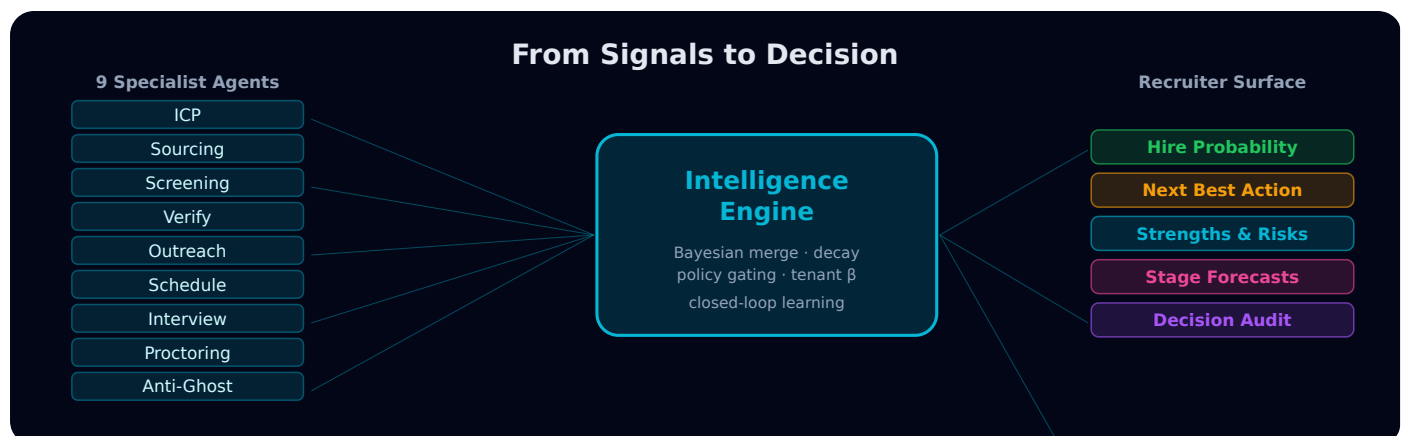


Figure 4 — Signal flow from agents through the Intelligence Engine to recruiter outputs

4.1 Composite score formula

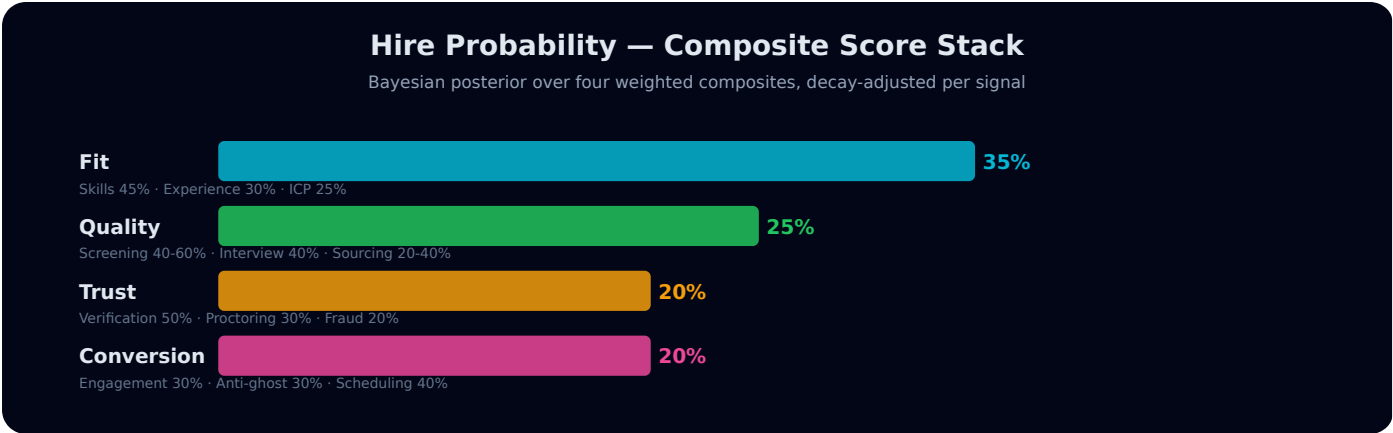


Figure 2 — Composite score weights feeding the Hire Probability gauge

For every candidate, the engine produces four sub-scores and a single Hire Probability:

Composite	Weight	Sub-weights
Fit Score	35%	Skill match 45% · Experience 30% · ICP pattern 25%
Quality Score	25%	Screening 40-60% · Interview 40% · Sourcing 20-40%
Trust Score	20%	Verification 50% · Proctoring integrity 30% · Fraud risk 20%
Conversion Score	20%	Outreach engagement 30% · Ghosting resistance 30% · Scheduling efficiency 40%

Sub-weights are dynamic: when a contributing agent hasn’t run yet, its weight is redistributed to the remaining contributors so the score is never penalized for missing data — it’s simply less confident (see §4.3).

4.2 Temporal decay

Every signal carries a timestamp. At read time it is decayed by an exponential half-life:

```
weight = 0.5 ^ (ageHours / halfLifeHours)
```

Half-lives are tuned per signal class:

Signal class	Half-life
Anti-Ghost (engagement freshness)	24 hours
Outreach reply velocity	48 hours
Interview performance	7 days
Screening / Sourcing match	30 days
ICP alignment	90 days

This means a hot candidate who replied yesterday outranks an identically-qualified candidate who replied last quarter — without us having to re-screen anyone.

4.3 Confidence breakdown

Every score ships with a confidence number computed from three factors:

```
confidence = 0.4 * completeness + 0.3 * freshness + 0.3 * criticalCoverage
```

- **Completeness:** fraction of expected agent signals present
- **Freshness:** weighted-average decay factor across all contributing signals
- **Critical coverage:** fraction of high-leverage signals (verification, interview, screening) that are present

Recruiters see confidence as a small badge next to each score. Low-confidence high-probability candidates surface a “needs more signal” tooltip.

4.4 Hire Probability — Bayesian formulation

The hire gauge is the posterior probability:

```
P(hire | signals) ∝ P(signals | hire) * P(hire)
```

Implemented as a logistic combiner over the four composites with tenant-trained coefficients:

```
logit(p) = β0 + βfit * fit + βquality * quality
          + βtrust * trust + βconversion * conversion
```

The β coefficients start at the platform-trained defaults and update per tenant via the closed-loop learning system (§7).

4.5 Stage-transition forecasts

Beyond hire probability, the engine emits three secondary forecasts:

- `nextStageSuccess` — probability the candidate clears the next pipeline stage
- `offerAcceptance` — probability they accept an offer if extended
- `dropoffProbability` — probability they ghost in the next 7 days

These power the “Lexy Candidate Prediction” plain-language summary on the candidate card.

4.6 Next Best Action waterfall

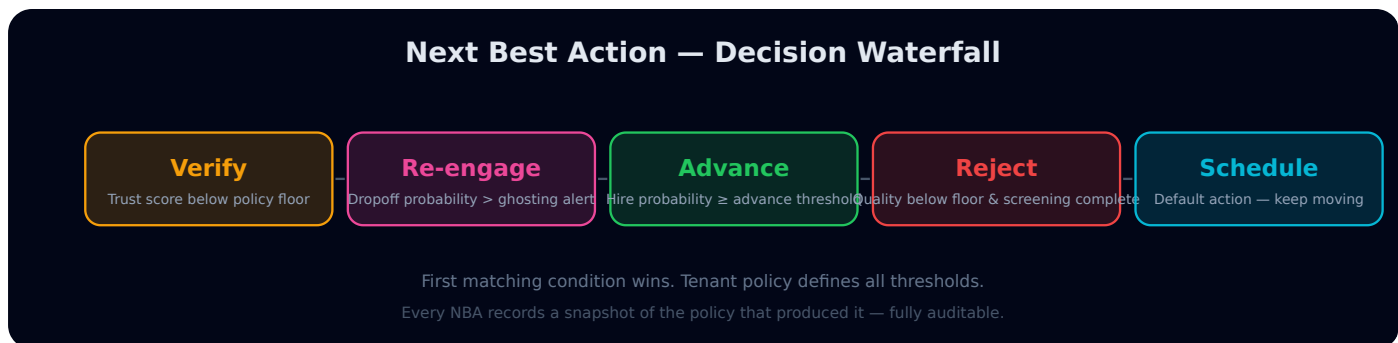


Figure 3 — Next Best Action waterfall (first match wins)

The recommendation engine is a deterministic waterfall over the scores and the tenant policy:

```
if trust < tenantPolicy.lowTrustThreshold
  → "Verify"
else if dropoffProbability > tenantPolicy.ghostingAlert
  → "Re-engage"
else if hireProbability >= tenantPolicy.advanceThreshold
  → "Advance"
else if quality < tenantPolicy.qualityFloor and screening_done
  → "Reject"
else
  → "Schedule"
```

The waterfall order is intentional: integrity issues short-circuit anything else, ghosting risk preempts advancement, and rejections require completed screening to avoid premature negative decisions.

5. The Connection Strength Engine

A separate engine (`lib/connectionEngine.ts`) tracks behavioral engagement signals distinct from fit:

Event	Δ Connection
Reply within 24h	+15
Schedules interview voluntarily	+20
Completes interview	+25
Asks a question back	+5
No-show	-25
Cancels < 2h before	-15
Goes silent > 5 days	-10

Connection strength feeds the Conversion composite and powers the Anti-Ghost tier-escalation logic. It's exposed as a signal on the candidate card so recruiters can see *why* a hot candidate has a high conversion score even if their fit is mid-tier.

6. Tenant decision policies

Every tenant configures a `DecisionPolicy` document (`lib/policies.ts`):

Field	Default	Purpose
<code>advanceThreshold</code>	80	Minimum hire probability to auto-advance
<code>qualityFloor</code>	40	Below this, suggest reject after screening
<code>lowTrustThreshold</code>	50	Force verification before any advance
<code>ghostingAlert</code>	60	Trigger re-engage NBA
<code>requireRecruiterApproval</code>	true	All advances must be human-confirmed
<code>lowTrustAction</code>	<code>"manual_verification"</code>	What to do on integrity flags
<code>autoRunAgents</code>	<code>[]</code>	Agents allowed to fire without recruiter trigger

Policies are versioned and every NBA decision records the policy snapshot that produced it, so the audit trail is reproducible even after a policy change.

7. Closed-loop learning

L3xy is not a static scoring model. Every recruiter override and every hiring outcome feeds back into the system.

7.1 Override capture

When a recruiter accepts, modifies, or rejects an NBA, the decision is appended to `overridesJson` on the candidate row with:

- The recommended action
- The recruiter's actual action
- The composite scores at decision time
- The policy snapshot
- Free-text reason (optional)

7.2 Outcome correlation

`routes/learning.ts` periodically computes per-tenant precision and recall metrics:

```
precision = hires_predicted_correctly / total_advance_recommendations
recall    = hires_predicted_correctly / total_hires
```

It also computes **Agent Coverage Impact** — hire rate for candidates who passed through each agent vs. hire rate for those who skipped it — which surfaces in the Analytics tab as recommendations like *“Lower your advance threshold to reduce manual review overhead.”*

7.3 Coefficient updates

The logistic `β` coefficients in §4.4 are re-fit per tenant nightly using the candidate population that has reached a terminal outcome (Hired or Rejected). This produces a tenant-specific scoring model without retraining the underlying LLMs.

7.4 Drift detection

`Override Rate` (recruiter disagreement %) is monitored continuously. A sustained spike triggers a calibration alert in the Analytics tab indicating the model has drifted relative to the tenant’s actual hiring behavior.

8. Data sources — the full waterfall

Sourcing is a strict three-tier waterfall. Tiers 1 and 2 are first-party and zero-cost. Tier 3 is external and paid, invoked only when the warm pools fall short.

8.1 Tier 1 — Tenant Talent Pool (always first)

Source	Role	Notes
<code>candidates</code> table	Tenant’s own historical pool	Re-scored against current ICP; signals decay-weighted
<code>role_fit_cache</code>	Per-job ICP match cache	Invalidated on ICP change, signal arrival, or policy update
<code>connection_events</code>	Engagement & conversation history	Carries forward — no need to re-warm
Verification cache	Prior trust signals	Re-used; no re-verification cost

Cost: **\$0**. Latency: **<200ms**. Should satisfy the majority of mature-tenant sourcing demand.

8.2 Tier 2 — Platform Talent Pool (cross-tenant, consented)

Source	Role	Notes
Platform federated index	Cross-tenant warm pool	Fit profiles only — no PII until candidate consent
Consent ledger	Per-candidate, per-tenant authorization	Append-only, fully auditable
Tenant policy (<code>platform_pool_share</code> / <code>_consume</code>)	Participation gates	Independent control per direction

Cost: **\$0** in API spend; introduction-request infrastructure required. Powers the two-sided talent flywheel that compounds with every new tenant.

8.3 Tier 3 — External Discovery (paid, last resort)

Source	Role	Notes
People Data Labs (PDL)	Primary 500M+ profile database	Elasticsearch DSL queries; <code>skills.name</code> flattening required
EnrichLayer	Phase-3b LinkedIn URL enricher	Output-side cap, per-URL failure logging
SerpAPI	Boolean LinkedIn discovery	Used when PDL coverage is sparse
GitHub API	Engineering-role discovery	<code>/search/users</code> with language + repo + follower filters
OpenAI / Anthropic	LLM inference (all tiers)	Routed through Replit-managed AI integrations

Within Tier 3, budget is spent in this priority order: GitHub (free) → SerpAPI → PDL → EnrichLayer (most expensive, applied to top candidates only).

8.4 Compounding economics

Every Tier 3 candidate that survives Screening is automatically promoted into Tier 1 (and, if policy allows, Tier 2). Sourcing cost-per-hire trends *down* as the tenant matures — the inverse of every conventional sourcing tool. Twelve-month mature tenants should be filling 60-80% of pipelines

from Tiers 1+2 alone, with external spend reserved for genuinely novel skill gaps and emerging markets.

9. Recruiter UX surfaces

9.1 Job dashboard

- WorkflowCanvas with the 9-stage agent strip in canonical order
- Real-time pipeline status, viable count, target progress
- Auto-run toggle for unattended pipelines

9.2 Candidate profile

- Intelligence Card (scores, gauge, NBA, strengths/risks, prediction)
- Decision Audit Trail with every signal and policy snapshot
- Stage-transition forecasts
- Conversation history across all channels

9.3 Outreach inbox

- Unified thread view across email/LinkedIn/SMS
- AI-drafted replies with sentiment analysis
- Connection strength inline per thread

9.4 Interview room

- Live AI interviewer with adaptive question selection
- Real-time transcript and STAR evaluation
- Proctoring integrity flags as they happen
- Recording + transcript stored against the candidate

9.5 Decision Queue

- Recruiter approval surface for “Advance” recommendations
- One-click accept / modify / reject with reason capture
- Bulk-action support for high-confidence batches

9.6 Analytics dashboard

- Funnel breakdown across all 9 stages
 - Bottleneck detection (median time per stage)
 - Calibration: precision/recall/override rate
 - Agent Coverage Impact
 - Learning Recommendations
-

10. Engineering “secret sauce”

A short list of design choices that distinguish L3xy from the competition:

1. **Two-phase sourcing with output-side cap.** Most competitors enrich every profile they discover, blowing the budget on dead-ends. L3xy only spends enrichment dollars on profiles that survive Phase-1 ranking.
2. **Additive signal merging, not replacement.** Each agent appends to a candidate’s signal vector — none overwrite prior signals. This means any agent can fire independently without losing historical context, and the engine can score a candidate even when only 3 of 9 agents have run.
3. **Decay-weighted confidence, not just decay-weighted scores.** Stale signals lose weight in the *score*, but they also lower the *confidence*, so recruiters see a real “this needs more data” signal rather than a deceptively crisp number.
4. **Dependency-graph pipeline builder.** The recruiter cannot construct an invalid pipeline (e.g. Verify without Screening). The UI auto-pulls prerequisites and persists the healed config — fixing not just the next run, but every legacy config saved before the rule existed.
5. **Per-tenant logistic coefficients.** The hire-probability model is platform-trained but tenant-calibrated. Two tenants hiring the same role can get different hire probabilities for the same candidate because their bar is different.
6. **NBA waterfall with policy snapshotting.** Every recommendation is reproducible from the snapshot — auditors and compliance teams can trace any decision to the exact policy state that produced it.
7. **Connection Strength as a first-class engine.** Behavioral signals (replies, no-shows, voluntary scheduling) are tracked separately from fit and feed the Conversion composite explicitly — making the difference between “qualified but cold” and “qualified and engaged” visible.

8. **Closed-loop coefficient updates without LLM retraining.** Nightly logistic re-fit is cheap, fast, and lets the system improve daily without paying for fine-tuning.

11. Roadmap (NDA partners)

- **Q3:** Greenhouse + Lever ATS bidirectional sync
- **Q3:** Native Google / Microsoft / Apple calendar
- **Q4:** Mobile recruiter app (Expo)
- **Q4:** Custom-agent SDK for tenant-supplied evaluators
- **Q4:** Take-home and system-design interview formats
- **Early next year:** Tenant-trained ICP foundation model (replacing per-job ICP prompt) for sub-second pipeline launch

12. Glossary

Term	Definition
ICP	Ideal Candidate Profile — structured spec extracted from a JD
NBA	Next Best Action — engine’s recommended action for a candidate
Composite Score	One of Fit / Quality / Trust / Conversion
Hire Probability	Bayesian posterior $P(\text{hire} \mid \text{signals})$
Connection Strength	Behavioral engagement score, distinct from Fit
Decision Audit Trail	Reproducible log of signals + policy → decision
Override	Recruiter accept/modify/reject of an NBA
Calibration Drift	Sustained increase in Override Rate signaling model staleness

L3xy Inc. — Confidential. *Distribution restricted to NDA signatories. Questions:*
engineering@l3xy.io · founders@l3xy.io